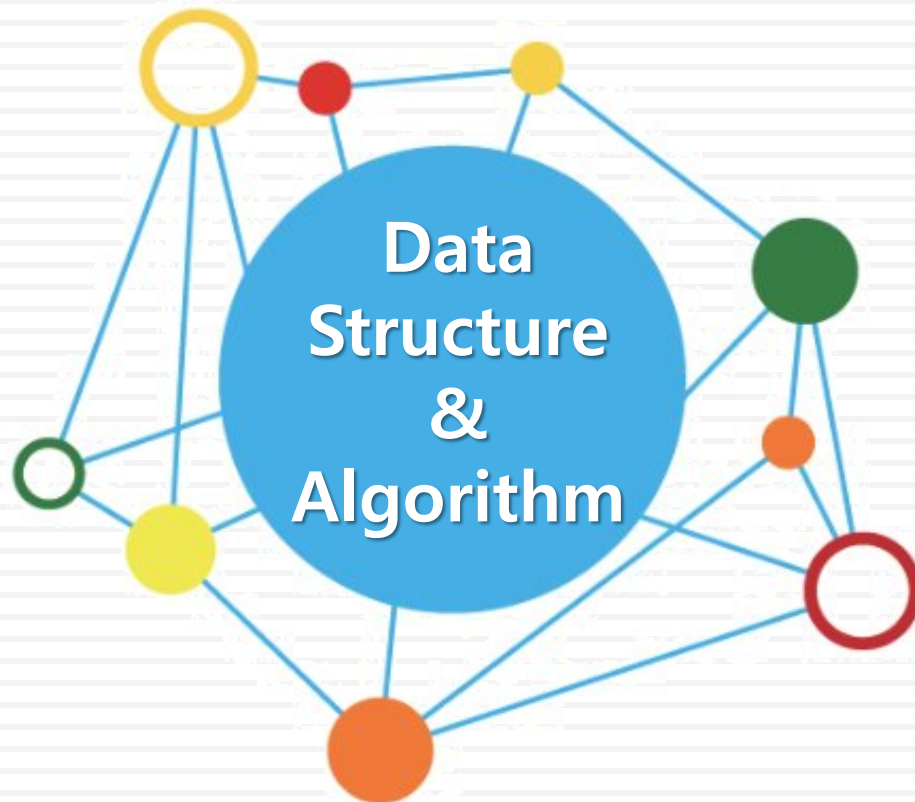


데이터 구조론



유길현

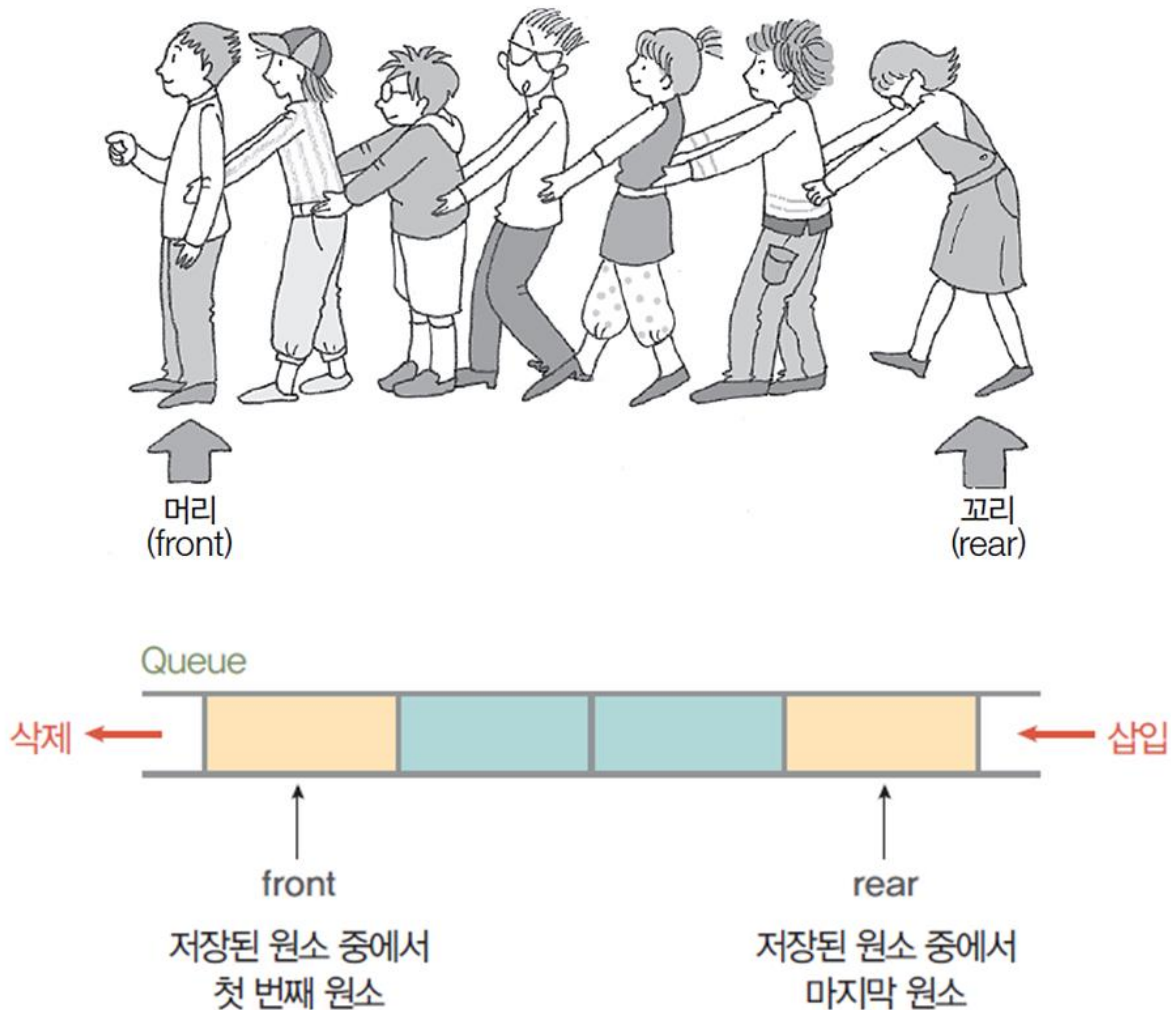
큐(Queue)

❖ 큐(Queue) 란?

- 먼저 들어온 데이터가 먼저 나가는 데이터구조
- 선입 선출(FIFO : First In First Out)



❖ 큐(Queue)의 구조



❖ 큐(Queue)의 연산

- 삽입 연산 : enQueue
- 삭제 연산 : deQueue

❖ 스택과 큐의 연산 비교

자료구조 \ 항목	삽입 연산		삭제 연산	
	연산자	삽입위치	연산자	삭제위치
스택	push	top	pop	top
큐	enQueue	rear	deQueue	front

ADT(Abstract Data Type)

큐의 추상 자료형

ADT Queue

데이터 : 0개 이상의 원소를 가진 유한 순서 리스트

연산 : $Q \in \text{Queue}$; $\text{item} \in \text{Element}$;

// 공백 큐를 생성 하는 연산

CreateQueue ::= create an empty Q;

// 큐가 공백인지 확인하는 연산

isEmpty(Q) ::= if(Q is empty) then return true
 else return false;

// 큐의 rear에 원소를 삽입하는 연산

enqueue(Q, item) ::= insert item at the rear of Q;

// 큐의 front에 있는 원소를 삭제하는 연산

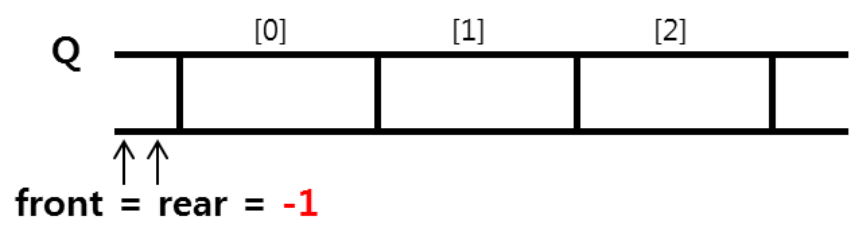
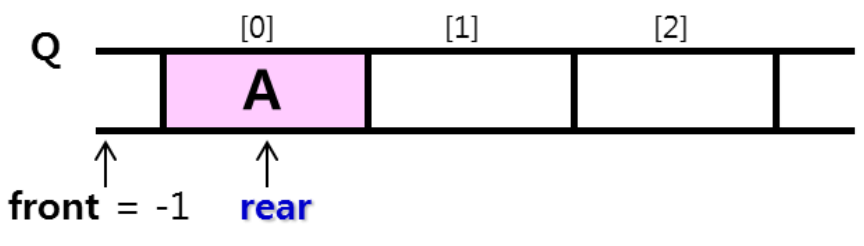
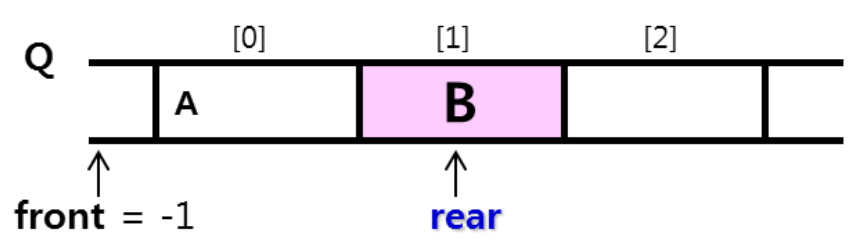
dequeue(s) ::= if(isEmpty(Q)) then return error
 else {delete and return the front item Q};

// 큐의 front에 있는 원소를 반환하는 연산

peek(S) ::= if(isEmpty(S)) then return error
 else {return the front item of the Q};

End Queue

큐(Queue)의 연산 과정

수행 순서	함수	큐의 동작
① 공백 큐 생성	<code>createQueue();</code>	 Q [0] [1] [2] ↑ ↑ front = rear = -1
② 원소 A 삽입	<code>enqueue(Q A);</code>	 Q [0] [1] [2] ↑ ↑ front = -1 rear
③ 원소 B 삽입	<code>enqueue(Q B);</code>	 Q [0] [1] [2] ↑ ↑ front = -1 rear

큐(Queue)의 연산 과정

수행 순서	함수	큐의 동작
④ 원소 삭제	deQueue(Q);	Q [0] [1] [2] A B ↑ ↑ front rear
⑤ 원소 C 삽입	enQueue(Q, C);	Q [0] [1] [2] B C ↑ ↑ front rear
⑥ 원소 삭제	deQueue(Q);	Q [0] [1] [2] B C ↑ ↑ front rear
⑦ 원소 삭제	deQueue(Q);	Q [0] [1] [2] C ↑ ↑ front rear

❖ 순차 큐

- 1차원 배열을 이용한 큐
 - 큐의 크기 = 배열의 크기
 - 변수 front : 저장된 첫 번째 원소의 인덱스 저장
 - 변수 rear : 저장된 마지막 원소의 인덱스 저장
- 상태 표현
 - 초기상태 : $\text{front} = \text{rear} = -1$
 - 공백상태 : $\text{front} = \text{rear}$
 - 포화상태 : $\text{rear} = n-1$ (n : 배열의 크기, $n-1$: 배열의 마지막 인덱스)

❖ 초기 공백 큐 생성 알고리즘

- 크기가 n 인 1차원 배열 생성
- front와 rear를 -1로 초기화

알고리즘	공백 순차 큐 생성
<pre>createQueue() Q[n]; front ← -1; rear ← -1; end createQueue()</pre>	

❖ 공백 큐 검사 알고리즘과 포화상태 검사 알고리즘

- 공백 상태 : $\text{front} = \text{rear}$
- 포화 상태 : $\text{rear} = n-1$ (n : 배열의 크기, $n-1$: 배열의 마지막 인덱스)

알고리즘	순차 큐 공백 상태 검사
<pre>isEmpty(Q) if(front == rear) then return true; else return false; end isEmpty()</pre>	

알고리즘	순차 큐의 포화 상태 검사
<pre>isFull(Q) if(rear == n - 1) then return true; else return false; end isFull()</pre>	

❖ 큐의 삽입 알고리즘

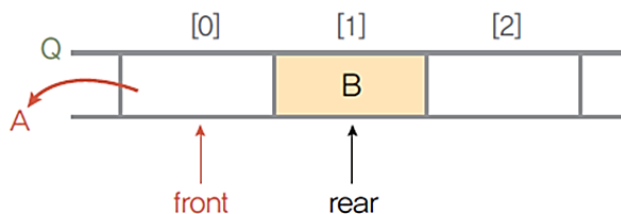
알고리즘	순차 큐의 원소 삽입
<pre>enqueue(Q, item) if(isFull(Q)) then Queue_Full(); // 포화 상태이면 삽입 연산 중단 else { ① rear ← rear + 1; ② Q[rear] ← item; } end enqueue()</pre>	

- 마지막 원소의 뒤에 삽입해야 하므로
 - ① 마지막 원소의 인덱스를 저장한 **rear**의 값을 하나 증가시켜 **삽입할 자리 준비**
 - ② 수정한 rear값에 해당하는 배열원소 Q[rear]에 item을 저장

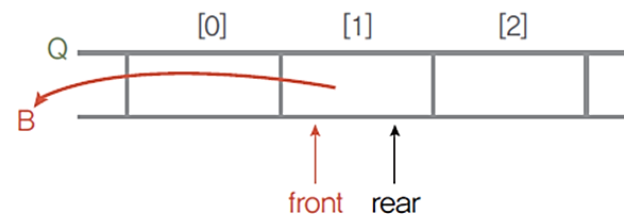
❖ 큐의 삭제 알고리즘

알고리즘	순차 큐의 원소 삭제
<pre>deQueue(Q) if(isEmpty(Q)) then Queue_Empty(); // 공백 상태이면 삽입 연산 중단 else { ① front ← front + 1; ② return Q[front]; } end deQueue()</pre>	

- 가장 앞에 있는 원소를 삭제해야 하므로
 - ① **front**의 위치를 이동하여 큐에 남아있는 첫 번째 원소의 위치로 이동하여 **삭제할 자리 준비**
 - ② front 자리의 원소를 삭제하여 반환
- deQueue() 연산 후 상태



첫 번째 deQueue() 연산 후 상태



두 번째 deQueue() 연산 후 상태

❖ 큐의 검색 알고리즘

알고리즘	순차 큐의 원소 검색
<pre>peek(Q) if(isEmpty(Q)) then Queue_Empty(); else return Q[front+1] end peek()</pre>	

- 가장 앞에 있는 원소를 검색하여 반환하는 연산
 - 현재 front의 한자리 뒤(front+1)에 있는 원소, 즉 큐에 있는 첫 번째 원소를 반환

❖ 순차 데이터구조 큐의 구조 정의

```
#define _CRT_SECURE_NO_WARNINGS
#include "stdio.h"
#include "stdlib.h"
#define Q_SIZE 4

// 큐 원소(element)의 자료형을 char로 정의
typedef char element;

typedef struct {
    element queue[Q_SIZE];    // 1차원 배열 큐 선언
    int front, rear;
} QueueType;
```

❖ 순차 데이터구조 공백 큐의 생성

```
// 공백 큐의 생성
QueueType *createQueue()
{
    QueueType *Q;
    Q=(QueueType *)malloc(sizeof(QueueType));
    Q->front=-1;
    Q->rear=-1;
    return Q;
};

// 순차 큐가 공백 상태인지 검사하는 연산
int isEmpty(QueueType *Q)
{
    if (Q->front == Q->rear){
        printf(" Queue is empty! ");
        return 1;
    }
    else
        return 0;
}
```

❖ 순차 데이터구조 큐의 포화상태 검사

```
// 순차 큐가 포화 상태인지 검사하는 연산
int isFull(QueueType *Q)
{
    if (Q->rear == Q_SIZE - 1) {
        printf(" Queue is full! ");
        return 1;
    }
    else
        return 0;
}

// 순차 큐의 rear에 원소를 삽입하는 연산
void enqueue(QueueType *Q, element item)
{
    if (isFull(Q))        // 포화 상태이면, 삽입 연산 중단
        return;
    else {
        Q->rear++;
        Q->queue[Q->rear] = item;
    }
}
```


❖ 순차 데이터구조 큐의 front에 원소를 삭제하는 연산

```
// 순차 큐의 front에서 원소를 삭제하는 연산
element dequeue(QueueType *Q)
{
    if (isEmpty(Q)) // 공백 상태이면, 삭제 연산 중단
        return 0;
    else
    {
        Q->front++;
        return Q->queue[Q->front];
    }
}
```

❖ 순차 데이터구조 큐의 원소를 검색, 출력하는 연산

```
// 순차 큐의 가장 앞에 있는 원소를 검색하는 연산
element peek(QueueType *Q)
{
    if (isEmpty(Q))    // 공백 상태이면 연산 중단
        exit(1);
    else
        return Q->queue[Q->front + 1];
}

// 순차 큐의 원소를 출력하는 연산
void printQ(QueueType *Q)
{
    int i;
    printf(" Queue : [");
    for (i = Q->front + 1; i <= Q->rear; i++)
        printf("%3c", Q->queue[i]);
    printf(" ]");
}
```

❖ 순차 데이터구조 큐의 동작을 확인

```
void main(void)
{
    QueueType *Q1 = createQueue(); // 큐 생성
    element data;
    printf("\n ***** 순차 큐 연산 ***** \n");
    printf("\n 삽입 A>>"); enqueue(Q1, 'A'); printQ(Q1);
    printf("\n 삽입 B>>"); enqueue(Q1, 'B'); printQ(Q1);
    printf("\n 삽입 C>>"); enqueue(Q1, 'C'); printQ(Q1);
    data = peek(Q1); printf(" peek item : %c \n", data);
    printf("\n 삭제 >>"); data = dequeue(Q1); printQ(Q1);
    printf("\t삭제 데이터: %c", data);
    printf("\n 삭제 >>"); data = dequeue(Q1); printQ(Q1);
    printf("\t삭제 데이터: %c", data);
    printf("\n 삭제 >>"); data = dequeue(Q1); printQ(Q1);
    printf("\t\t삭제 데이터: %c", data);

    printf("\n 삽입 D>>"); enqueue(Q1, 'D'); printQ(Q1);
    printf("\n 삽입 E>>"); enqueue(Q1, 'E'); printQ(Q1);
    getchar();
}
```

❖ 순차 데이터 구조를 이용한 큐 프로그램 : [교재 280p](#)

■ 실행 결과

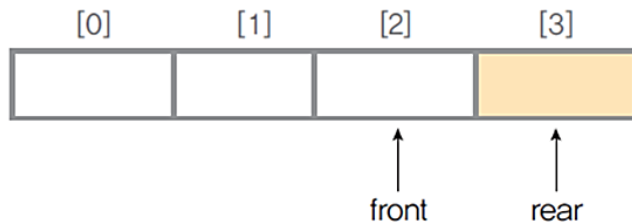
```
C:\한국산업기술대학\2021년1학기\데이터구조론\실습예제\큐\Queue)...
***** 순차 큐 연산 *****
삽입 A>> Queue : [ A ]
삽입 B>> Queue : [ A B ]
삽입 C>> Queue : [ A B C ] peek item : A

삭제 >> Queue : [ B C ]      삭제 데이터 : A
삭제 >> Queue : [ C ]        삭제 데이터 : B
삭제 >> Queue : [ ]          삭제 데이터 : C
삽입 D>> Queue : [ D ]
삽입 E>> Queue is full! Queue : [ D ]

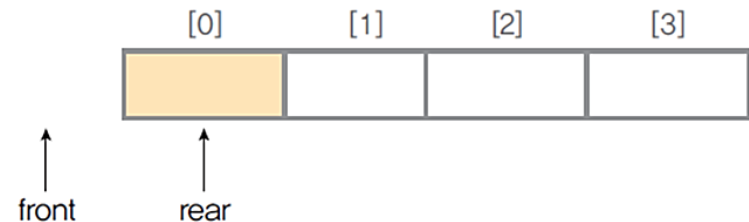
-----
Process exited after 14.12 seconds with return value 10
계속하려면 아무 키나 누르십시오 . . .
```

❖ 순차 큐의 잘못된 포화 인식

- 큐에서 삽입과 삭제를 반복하면서 그림(a)와 같은 상태일 경우, 앞부분에 빈자리가 있지만 $rear=n-1$ 상태이므로 포화상태로 인식하고 더 이상의 삽입을 수행하지 않는다
- 순차 큐의 잘못된 포화상태 인식의 해결 방법-1
 - 저장된 원소들을 배열의 앞부분으로 이동 시키기
 - ☞ 순차자료에서의 이동 작업은 연산이 복잡하여 효율성이 떨어짐



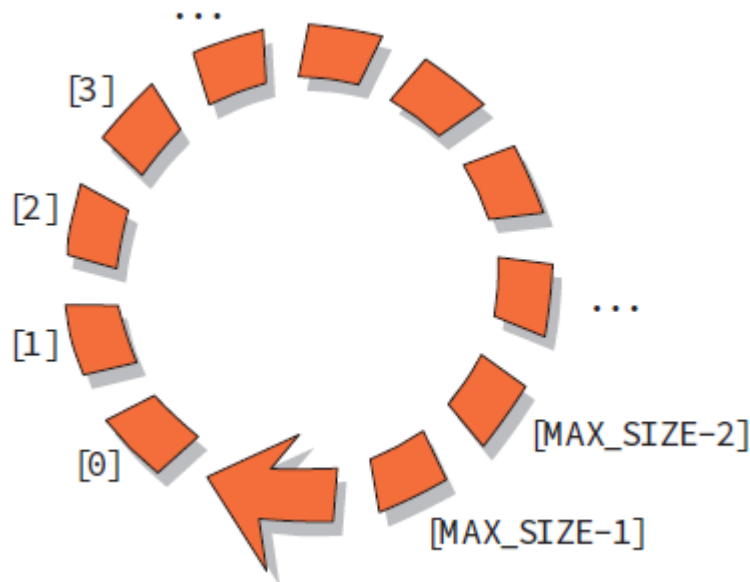
포화 상태로 잘못 인식하는 경우



큐의 원소들을 앞으로 이동하여 해결

❖ 순차 큐의 잘못된 포화상태 인식의 해결 방법-2

- 1차원 배열을 사용 하면서 논리적으로 배열의 처음과 끝이 연결되어 있다고 가정하고 사용 $\Rightarrow$ 원형 큐
- 원형 큐의 논리적 구조



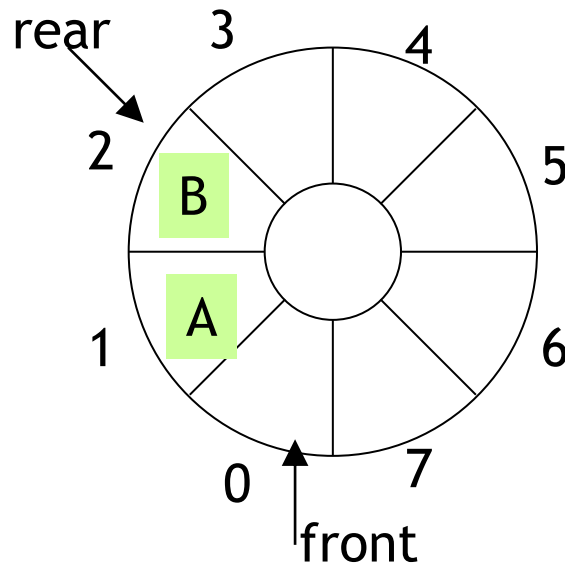
❖ 원형 큐의 구조

- 큐의 전단과 후단을 관리하기 위한 2개의 변수 필요
- 초기 공백 상태 : $\text{front} = \text{rear} = 0$
- front 와 rear 의 위치가 배열의 마지막 인덱스 $n-1$ 에서 논리적인 다음 자리인 인덱스 0번으로 이동하기 위해서 **나머지연산자 mod**를 사용
 - $3 \div 4 = 0 \dots 3$ (몫=0, 나머지=3)
 - $3 \bmod 4 = 3$
- 공백 상태와 포화 상태 구분을 쉽게 하기 위해서 front 가 있는 자리는 사용하지 않고 항상 빈자리로 둬
- 순차 큐와 원형 큐의 비교

종류	삽입 위치	삭제 위치
순차 큐	$\text{rear} = \text{rear} + 1$	$\text{front} = \text{front} + 1$
원형 큐	$\text{rear} = (\text{rear} + 1) \bmod n$	$\text{front} = (\text{front} + 1) \bmod n$

❖ 원형 큐의 구조

- front : 첫 번째 요소 하나 앞의 인덱스
- rear : 마지막 요소의 인덱스



❖ 초기 공백 원형 큐 생성 알고리즘

- 크기가 n 인 1차원 배열 생성
- front와 rear를 0으로 초기화

알고리즘	공백 원형 큐 생성
<pre>createQueue() cQ[n]; front $\leftarrow$ 0; rear $\leftarrow$ 0; end createQueue()</pre>	

순차 데이터 구조를 이용한 원형 큐의 구현

❖ 원형 큐의 공백상태 검사 알고리즘 포화상태 검사 알고리즘

알고리즘	원형 큐의 공백 상태검사
<pre>isEmpty(cQ) if (front == rear) then return true; else return false; end isEmpty()</pre>	

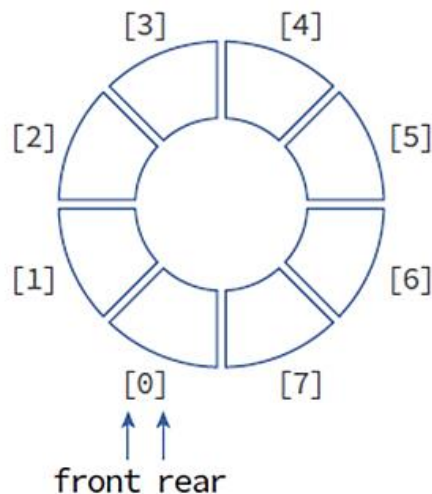
알고리즘	원형 큐의 포화 상태검사
<pre>isFull(cQ) if(((rear+1) mod n) = front) then return true; else return false; end isFull()</pre>	

■ 원형 큐의 상태에 따른 front와 rear의 관계

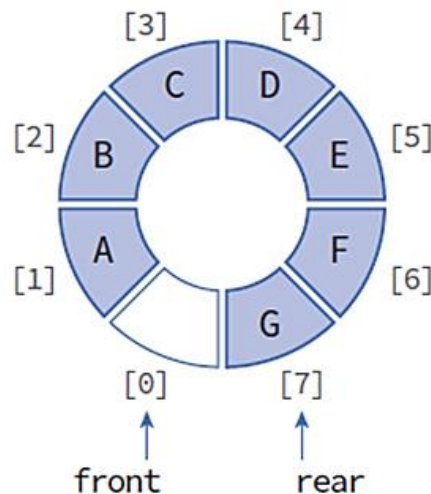
구분	조건
공백 상태	$front = rear$
포화 상태	$(rear+1) \bmod n = front$

❖ 원형 큐의 공백상태와 포화상태

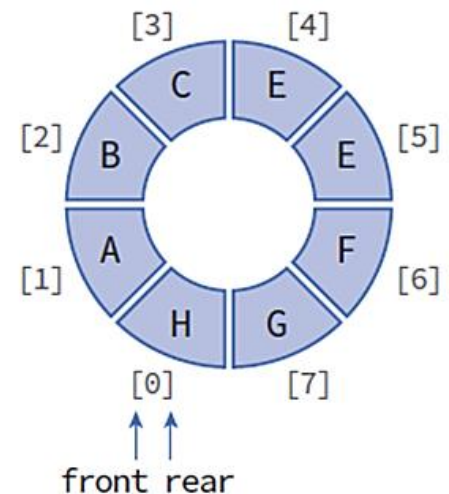
- 공백 상태 : $front == rear$
- 포화 상태 : $front \% M == (rear + 1) \% M$
- 공백상태와 포화상태를 구별하기 위하여 하나의 공간은 항상 비워둔다.



공백 상태



포화 상태



오류 상태

❖ 원형 큐의 삽입 알고리즘

알고리즘	원형 큐의 원소 삽입
<pre>enQueue(cQ, item) if (isFull(cQ)) then Queue_Full(); // 포화 상태이면 삽입 연산 중단 else { ① rear ← (rear + 1) mod n; ② cQ[rear] ← item; } end enQueue()</pre>	

① rear의 값을 조정하여 삽입할 자리를 준비

- $\text{rear} \leftarrow (\text{rear} + 1) \text{ mode } n$

② 준비한 자리 cQ[rear]에 원소 item을 삽입

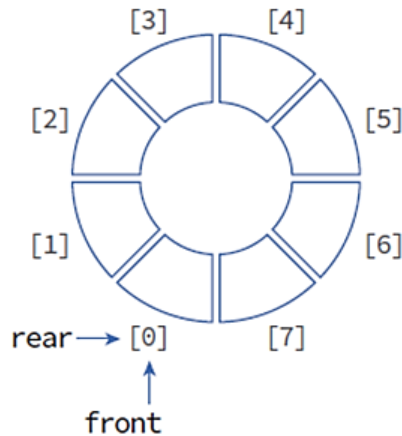
❖ 원형 큐의 삭제 알고리즘

알고리즘	원형 큐의 원소 삭제
<pre>deQueue(cQ) if (isEmpty(cQ)) then Queue_Empty(); // 공백 상태이면 삽입 연산 중단 else { ① front ← (front + 1) mod n; ② return cQ[front]; } end deQueue()</pre>	

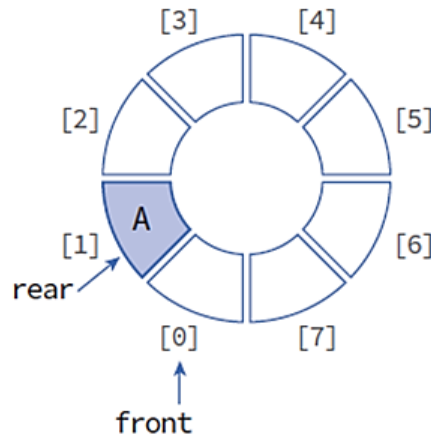
- ① front의 값을 조정하여 삭제할 자리를 준비
- ② 준비한 자리에 있는 원소 cQ[front]를 삭제하여 반환

순차 데이터 구조를 이용한 원형 큐의 구현

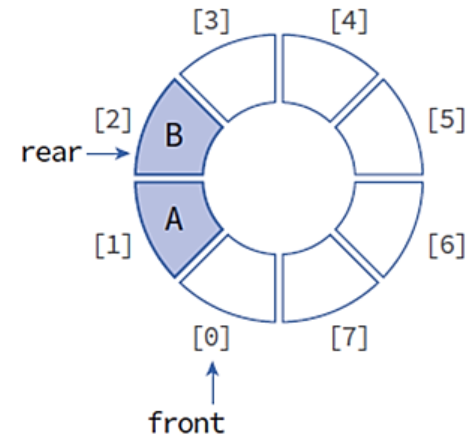
❖ 원형 큐의 동작



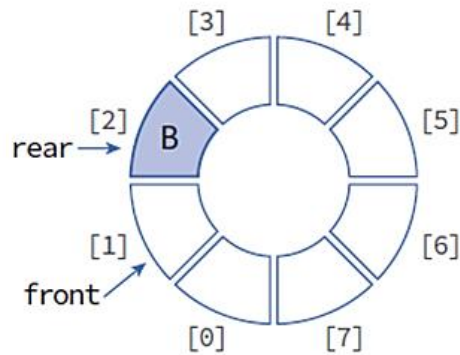
초기 상태



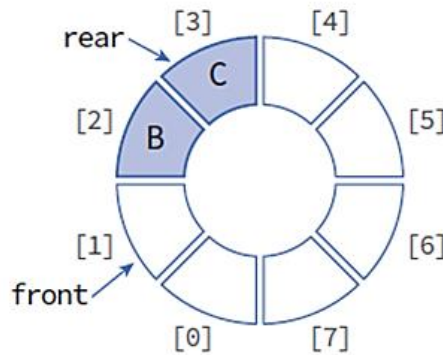
A 삽입



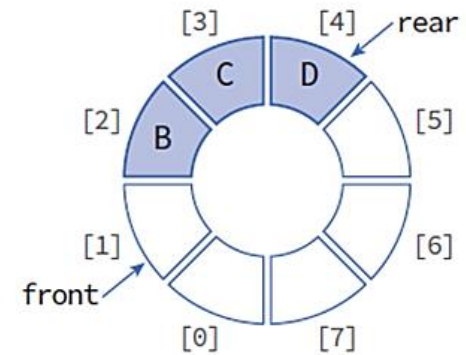
B 삽입



삭제



C 삽입



D 삽입

❖ 순차 데이터 구조 원형 큐의 구조 정의

```
#define _CRT_SECURE_NO_WARNINGS
#include "stdio.h"
#include "stdlib.h"
#define cQ_SIZE 4

typedef char element;    // 큐 원소(element)의 자료형을 char로 정의

typedef struct {
    element queue[cQ_SIZE]; // 1차원 배열 큐 선언
    int front, rear;
} QueueType;
```

❖ 순차 데이터 구조 공백 원형 큐의 생성

```
// 공백 원형 큐의 생성
QueueType *createQueue()
{
    QueueType *cQ;
    cQ = (QueueType *)malloc(sizeof(QueueType));
    cQ->front = 0;        // front 초기값 설정
    cQ->rear = 0;         // rear 초기값 설정
    return cQ;
}

// 원형 큐가 공백 상태인지 검사하는 연산
int isEmpty(QueueType *cQ)
{
    if (cQ->front == cQ->rear) {
        printf(" Circular Queue is empty! ");
        return 1;
    }
    else
        return 0;
}
```


❖ 순차 데이터 구조 원형 큐의 포화상태 검사

```
// 원형 큐가 포화 상태인지 검사하는 연산
int isFull(QueueType *cQ)
{
    if (((cQ->rear + 1) % cQ_SIZE) == cQ->front)
    {
        printf(" Circular Queue is full! ");
        return 1;
    }
    else
        return 0;
}
```

❖ 순차 데이터 구조 원형 큐의 원소 삽입 연산

```
// 원형 큐의 rear에 원소를 삽입하는 연산
void enqueue(QueueType *cQ, element item)
{
    if (isFull(cQ))
        return;
    else {
        cQ->rear = (cQ->rear + 1) % cQ_SIZE;
        cQ->queue[cQ->rear] = item;
    }
}

// 원형 큐의 front에서 원소를 삭제하고 반환하는 연산
element dequeue(QueueType *cQ)
{
    if (isEmpty(cQ))
        exit(1);
    else {
        cQ->front = (cQ->front + 1) % cQ_SIZE;
        return cQ->queue[cQ->front];
    }
}
```

❖ 순차 데이터 구조 원형 큐의 원소를 검색

```
// 원형 큐의 가장 앞에 있는 원소를 검색하는 연산
element peek(QueueType *cQ)
{
    if (isEmpty(cQ))
        exit(1);
    else
        return cQ->queue[(cQ->front + 1) % cQ_SIZE];
}
```

❖ 순차 데이터 구조 원형 큐의 원소를 출력

```
// 원형 큐의 원소를 출력하는 연산
void printQ(QueueType *cQ)
{
    int i, first, last;
    first = (cQ->front + 1) % cQ_SIZE;
    last = (cQ->rear + 1) % cQ_SIZE;
    printf(" Circular Queue : [");
    i = first;
    while (i != last)
    {
        printf("%3c", cQ->queue[i]);
        i = (i + 1) % cQ_SIZE;
    }
    printf(" ] ");
}
```

❖ 순차 데이터 구조 원형 큐의 수행을 확인

```
void main(void)
{
    QueueType *cQ = createQueue(); // 큐 생성
    element data;
    printf("\n ***** 원형 큐 연산 ***** \n");
    printf("\n 삽입 A>>"); enqueue(cQ, 'A'); printQ(cQ);
    printf("\n 삽입 B>>"); enqueue(cQ, 'B'); printQ(cQ);
    printf("\n 삽입 C>>"); enqueue(cQ, 'C'); printQ(cQ);
    data = peek(cQ); printf(" peek item : %c \n", data);
    printf("\n 삭제 >>"); data = dequeue(cQ); printQ(cQ);
    printf("\t삭제 데이터: %c", data);
    printf("\n 삭제 >>"); data = dequeue(cQ); printQ(cQ);
    printf("\t삭제 데이터: %c", data);
    printf("\n 삭제 >>"); data = dequeue(cQ); printQ(cQ);
    printf("\t\t삭제 데이터: %c", data);
    printf("\n 삽입 D>>"); enqueue(cQ, 'D'); printQ(cQ);
    printf("\n 삽입 E>>"); enqueue(cQ, 'E'); printQ(cQ);
    getchar();
}
```

순차 데이터 구조를 이용한 원형 큐의 구현 프로그램

❖ 순차 데이터 구조를 이용한 원형 큐 프로그램 : [교재 287p](#)

■ 실행 결과

```
C:\한국산업기술대학\2021년1학기\데이터구조론\실습예제\큐\Queue)...
***** 원형 큐 연산 *****

삽입 A>> Circular Queue : [ A ]
삽입 B>> Circular Queue : [ A B ]
삽입 C>> Circular Queue : [ A B C ]  peek item : A

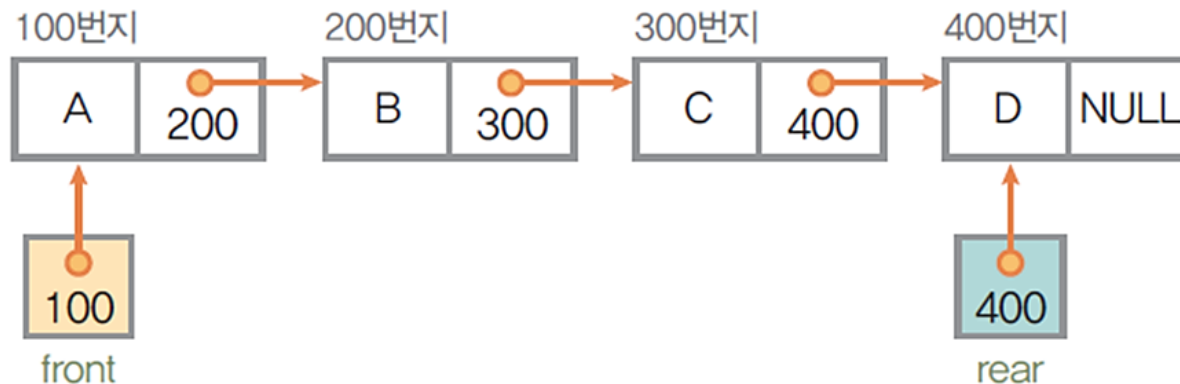
삭제  >> Circular Queue : [ B C ]      삭제 데이터: A
삭제  >> Circular Queue : [ C ]        삭제 데이터: B
삭제  >> Circular Queue : [ ]          삭제 데이터: C

삽입 D>> Circular Queue : [ D ]
삽입 E>> Circular Queue : [ D E ]

-----
Process exited after 4.514 seconds with return value 10
계속하려면 아무 키나 누르십시오 . . .
```

❖ 단순 연결 리스트를 이용한 큐

- 큐의 원소 : 단순 연결 리스트의 노드
- 큐의 원소의 순서 : 노드의 링크를 포인터로 연결
- 변수 front : 첫 번째 노드를 가리키는 포인터 변수
- 변수 rear : 마지막 노드를 가리키는 포인터 변수
- 상태 표현
 - 초기 상태와 공백 상태 : $\text{front} = \text{rear} = \text{null}$
- 연결 큐의 구조



❖ 공백 연결 큐 생성 알고리즘

- 초기화 : front = rear = null

알고리즘	공백 연결 큐 생성
<pre>createLinkedQueue() front ← NULL; rear ← NULL; end createLinkedQueue()</pre>	

❖ 연결 큐의 공백 상태 검사 알고리즘

- 공백 상태 : front = rear = null

알고리즘	공백 연결 큐 생성
<pre>isEmpty(LQ) if (front == NULL) then return true; else return false; end isEmpty()</pre>	

❖ 연결 큐의 삽입 알고리즘

알고리즘	연결 큐의 원소 삽입
<pre>enQueue(LQ, item) ① { new ← getNode(); { new data ← item; { new link ← NULL; ② { if (front == NULL) { { rear ← new; { front ← new; } ③ { else { { rear.link ← new; { rear ← new; } } } end enQueue()</pre>	

❖ 연결 큐의 삽입 알고리즘

- ① 삽입할 새 노드를 생성하여 데이터 필드에 item을 저장. 삽입할 새 노드는 연결 큐의 마지막 노드가 되어야 하므로 링크 필드에 NULL을 저장

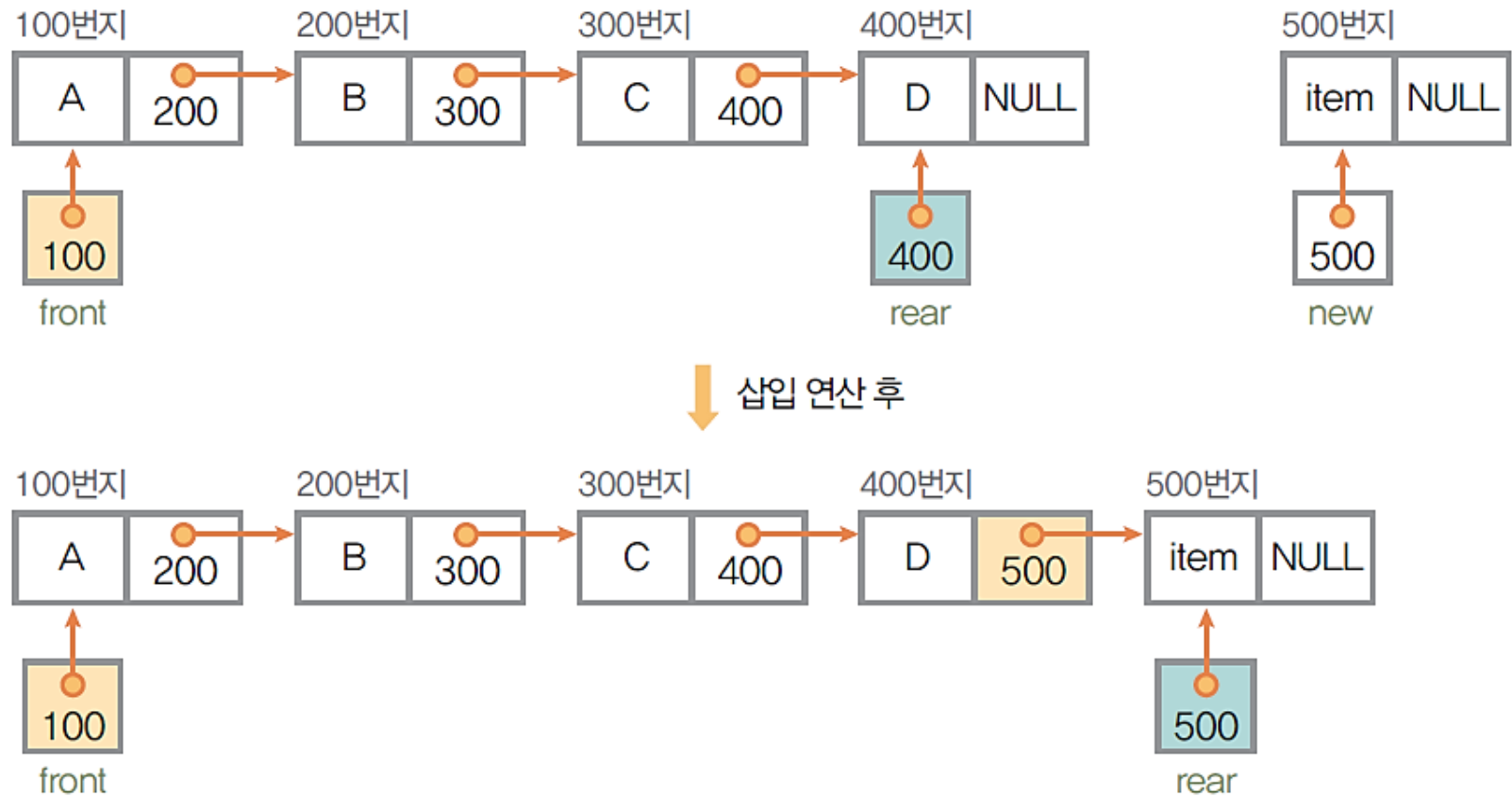


- ② 새 노드를 삽입하기 전에 연결 큐가 공백인지 아닌지를 검사. 연결 큐가 공백인 경우에는 삽입할 새 노드가 큐의 첫 번째 노드이자 마지막 노드이므로 포인터 front와 rear가 모두 새 노드를 가리키도록 설정



❖ 연결 큐의 삽입 알고리즘

- ③ 큐가 공백이 아닌 경우, 즉 노드가 있는 경우에는 현재 큐의 마지막 노드의 뒤에 새 노드를 삽입하고 마지막 노드를 가리키는 rear가 삽입한 새 노드를 가리키도록 설정

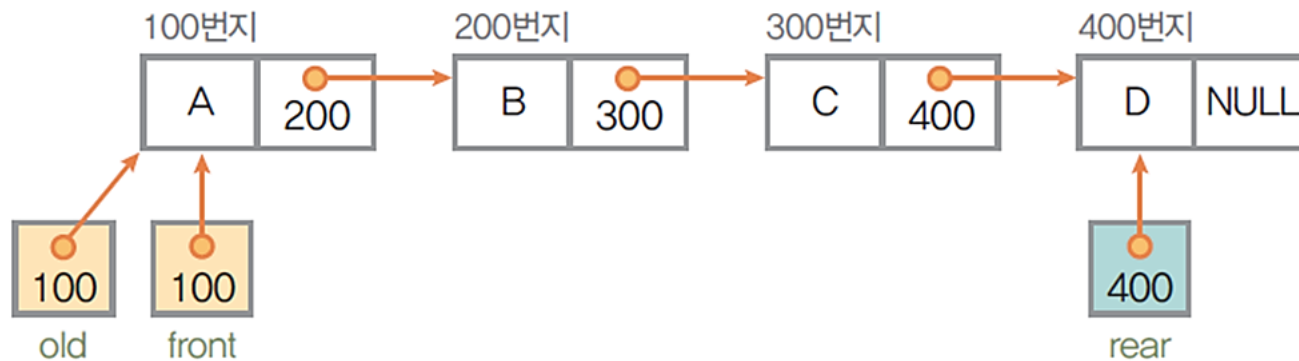


❖ 연결 큐의 원소 삭제 알고리즘

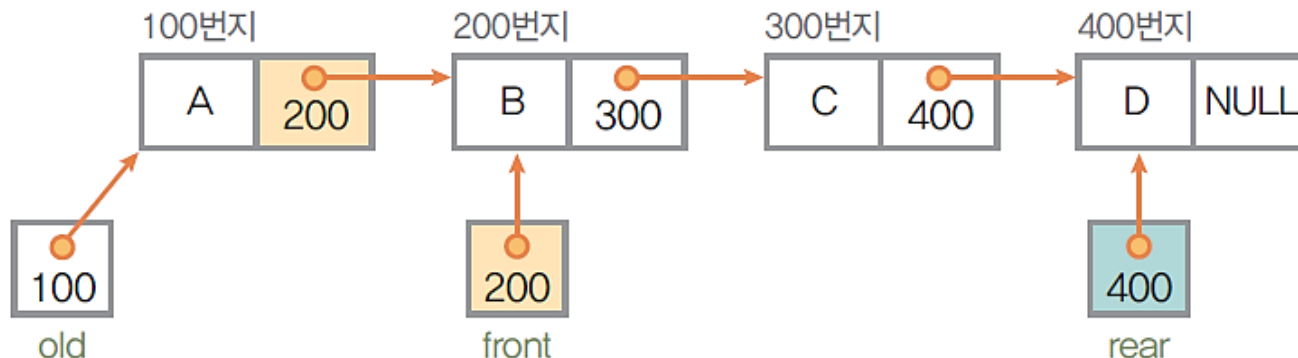
알고리즘	연결 큐의 원소 삭제
<pre>deQueue(LQ) if (isEmpty(LQ) then Queue_Empty(); else { ① old ← front; item ← front.data; ② front ← front.link; ③ if (isEmpty(LQ)) then rear ← NULL; ④ returnNode(old); return item; } end deQueue()</pre>	

❖ 연결 큐의 원소 삭제 알고리즘

- ① 삭제 연산에서 삭제할 노드는 큐의 첫 번째 노드로, 포인터 front 가 가리키고 있는 노드. Front가 가리키는 노드를 포인터 old가 가리키게 하여 삭제할 노드로 지정

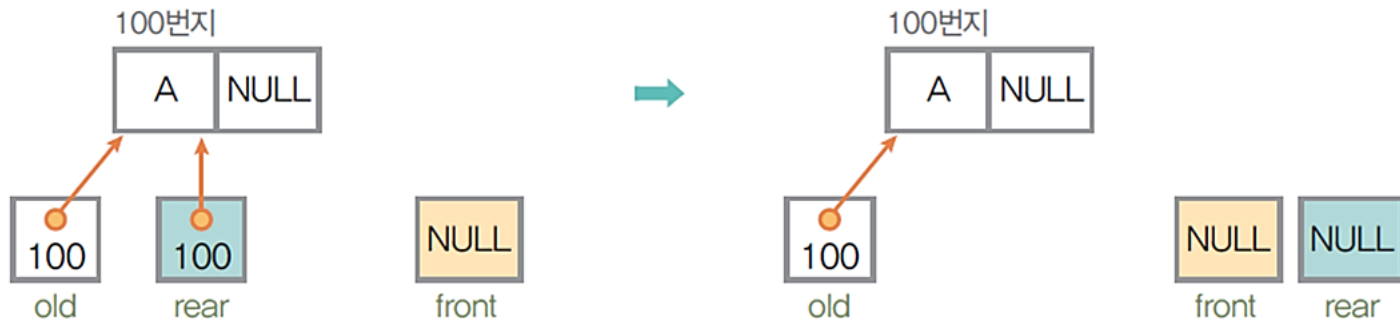


- ② 삭제 연산 후에는 현재 front 노드 다음 노드(front.link)가 front 노드가 되어야 하므로 포인터 front를 재설정

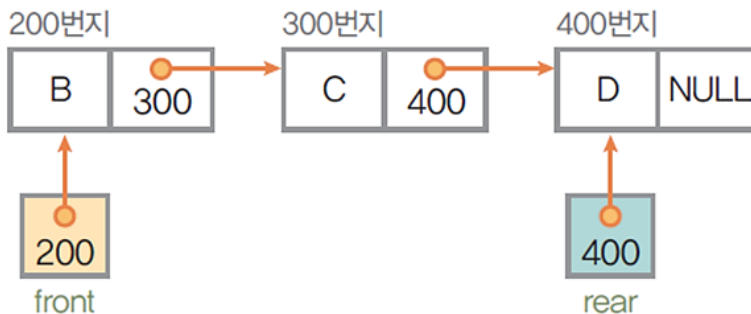


❖ 연결 큐의 원소 삭제 알고리즘

- ③ 현재 큐에 노드가 하나뿐이어서 재설정된 front가 NULL이 되는 경우에는 삭제 연산 후에 공백 큐가 되므로 포인터 rear를 NULL로 설정



- ④ 포인터 old가 가리키고 있는 노드를 삭제하여 메모리 공간을 시스템에 반환(returnNode())



❖ 연결 큐의 원소 검색 알고리즘

- 연결 큐의 첫 번째 노드, 즉 front 노드의 데이터 필드 값을 반환

알고리즘	연결 큐의 원소 검색
<pre>peek(LQ) if (isEmpty(LQ) then Queue_Empty(); else return (front.data); end peek()</pre>	

❖ 연결 큐의 자료형 및 노드 구조 정의

```
#define _CRT_SECURE_NO_WARNINGS
#include "stdio.h"
#include "stdlib.h"

// 연결 큐 원소(element)의 자료형을 char로 정의
typedef char element;

// 연결 큐의 노드를 구조체로 정의
typedef struct QNode {
    element data;
    struct QNode *link;
} QNode;

// 연결 큐에서 사용하는 포인터 front와 rear를 구조체로 정의
typedef struct {
    QNode *front, *rear;
} LQueueType;
```


❖ 연결 큐의 생성과 공백상태를 검사하는 연산

```
// 공백 연결 큐를 생성하는 연산
LQueueType *createLinkedQueue()
{
    LQueueType *LQ;
    LQ = (LQueueType *)malloc(sizeof(LQueueType));
    LQ->front = NULL;
    LQ->rear = NULL;
    return LQ;
}

// 연결 큐가 공백 상태인지 검사하는 연산
int isEmpty(LQueueType *LQ)
{
    if (LQ->front == NULL) {
        printf(" Linked Queue is empty! ");
        return 1;
    }
    else
        return 0;
}
```

❖ 연결 큐에 원소를 삽입하는 연산

```
// 연결 큐에 원소를 삽입하는 연산
void enqueue(LQueueType *LQ, element item)
{
    QNode *newNode = (QNode *)malloc(sizeof(QNode));
    newNode->data = item;
    newNode->link = NULL;
    if (LQ->front == NULL)    // 현재 연결 큐가 공백 상태인 경우
    {
        LQ->front = newNode;
        LQ->rear = newNode;
    }
    else                      // 현재 연결 큐가 공백 상태가 아닌 경우
    {
        LQ->rear->link = newNode;
        LQ->rear = newNode;
    }
}
```

❖ 연결 큐에서 원소를 삭제하고 반환하는 연산

```
// 연결 큐에서 원소를 삭제하고 반환하는 연산
element deQueue(LQueueType *LQ)
{
    QNode *old = LQ->front;
    element item;
    if (isEmpty(LQ))
        return 0;
    else {
        item = old->data;
        LQ->front = LQ->front->link;
        if (LQ->front == NULL)
            LQ->rear = NULL;
        free(old);
        return item;
    }
}
```

❖ 연결 큐에서 원소를 검색하는 연산

```
// 연결 큐에서 원소를 검색하는 연산
element peek(LQueueType *LQ)
{
    element item;
    if (isEmpty(LQ))
        return 0;
    else {
        item = LQ->front->data;
        return item;
    }
}
```

❖ 연결 큐의 원소를 출력하는 연산

```
// 연결 큐의 원소를 출력하는 연산
void printLQ(LQueueType *LQ)
{
    QNode *temp = LQ->front;
    printf(" Linked Queue : (");
    while (temp) {
        printf("%3c", temp->data);
        temp = temp->link;
    }
    printf(" ] ");
}
```

❖ 연결 큐의 동작을 확인

```
void main(void)
{
    LQueueType *LQ = createLinkedQueue(); // 연결 큐 생성
    element data;
    printf("\n ***** 연결 큐 연산 ***** \n");
    printf("\n 삽입 A>>"); enqueue(LQ, 'A'); printLQ(LQ);
    printf("\n 삽입 B>>"); enqueue(LQ, 'B'); printLQ(LQ);
    printf("\n 삽입 C>>"); enqueue(LQ, 'C'); printLQ(LQ);
    data = peek(LQ); printf(" peek item : %c \n", data);
    printf("\n 삭제 >>"); data = dequeue(LQ); printLQ(LQ);
    printf("\t삭제 데이터: %c", data);
    printf("\n 삭제 >>"); data = dequeue(LQ); printLQ(LQ);
    printf("\t삭제 데이터: %c", data);
    printf("\n 삭제 >>"); data = dequeue(LQ); printLQ(LQ);
    printf("\t\t삭제 데이터: %c", data);
    printf("\n 삽입 D>>"); enqueue(LQ, 'D'); printLQ(LQ);
    printf("\n 삽입 E>>"); enqueue(LQ, 'E'); printLQ(LQ);
    getchar();
}
```

❖ 연결 데이터 구조를 이용해 연결 큐 구현하기 프로그램 : [교재 297p](#)

■ 실행 결과

```
C:\한국산업기술대학교\2021년1학기\데이터구조론\실습예제\큐(Queue)...  _  □  X

***** 연결 큐 연산 *****

삽입 A>> Linked Queue : [ A ]
삽입 B>> Linked Queue : [ A B ]
삽입 C>> Linked Queue : [ A B C ]  peek item : A

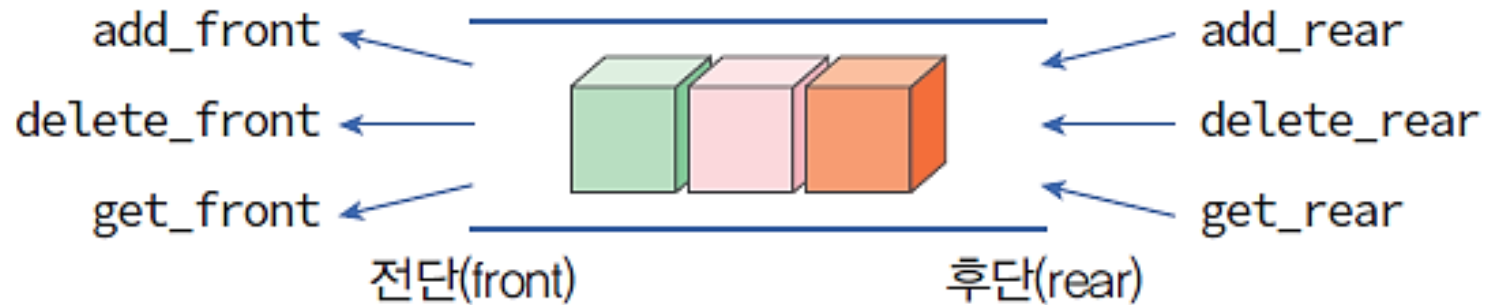
삭제  >> Linked Queue : [ B C ]      삭제 데이터 : A
삭제  >> Linked Queue : [ C ]        삭제 데이터 : B
삭제  >> Linked Queue : [ ]          삭제 데이터 : C

삽입 D>> Linked Queue : [ D ]
삽입 E>> Linked Queue : [ D E ]

-----
Process exited after 3.515 seconds with return value 10
계속하려면 아무 키나 누르십시오 . . .
```

❖ 데크(Deque : double-ended queue)

- 데크는 double-ended queue의 줄임말로 큐의 전단(front)과 후단(rear)에서 삽입과 삭제가 가능한 큐



❖ 데크의 추상 자료형

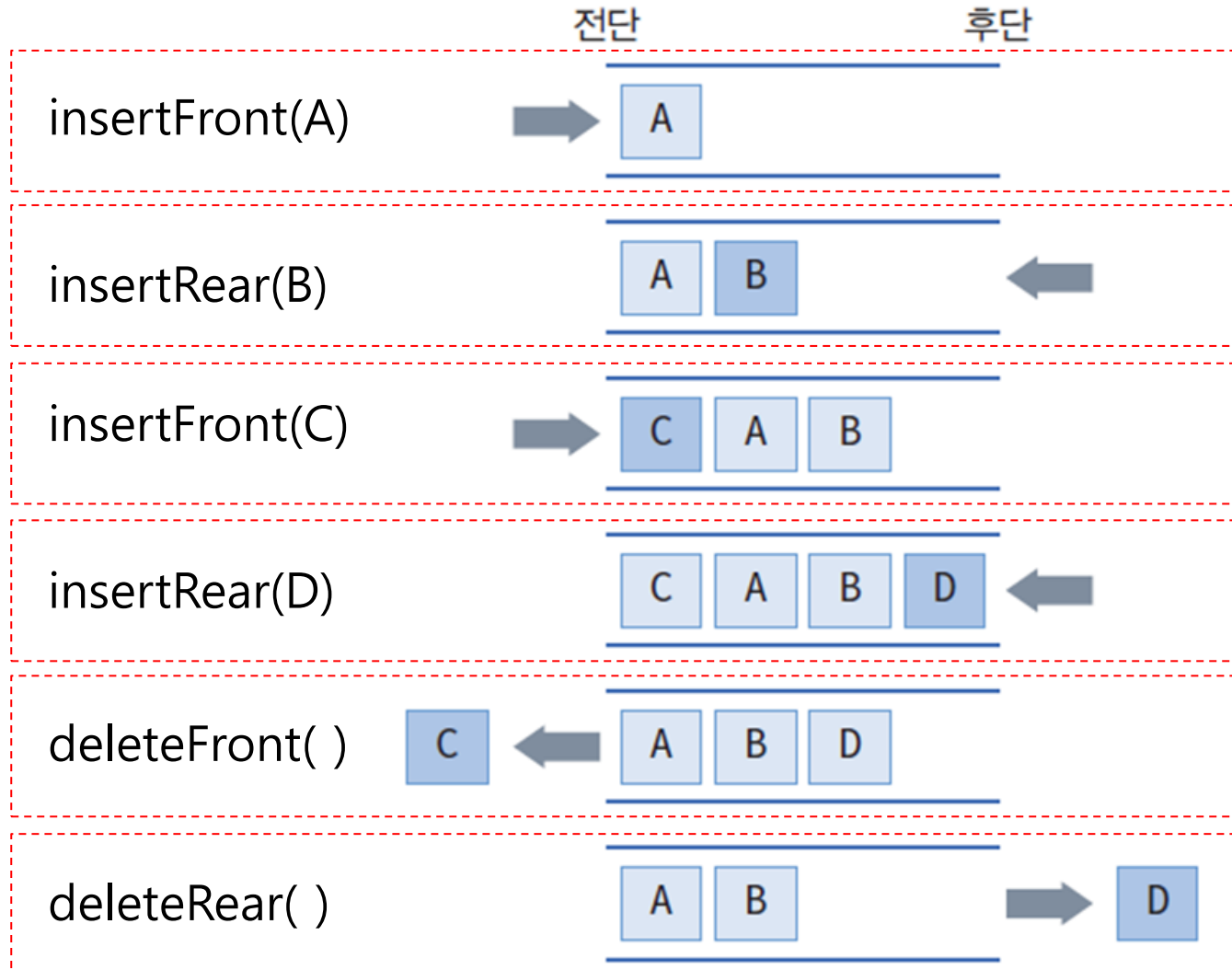
ADT	데크의 추상 자료형
ADT deque 데이터 : 0개 이상의 원소를 가진 유한 순서 리스트 연산 : $DQ \in \text{deque}; \text{item} \in \text{Element};$ // 공백 데크을 생성하는 연산 createDeque () ::= create an empty DQ; // 데크가 공백인지 아닌지를 확인하는 연산 isEmpty (DQ) ::= if (DQ is empty) then return true else return false; // 데크의 front 앞에 item(원소)을 삽입하는 연산 insertFront (DQ, item) ::= insert item at the front of DQ; // 데크의 rear 뒤에 item(원소)을 삽입하는 연산 insertRear (DQ, item) ::= insert item at the rear of DQ;	

❖ 데크의 추상 자료형

ADT	데크의 추상 자료형
	<pre>// 데크의 front에 있는 item(원소)을 데크에서 삭제하고 반환하는 연산 deleteFront(DQ) ::= if (isEmpty(DQ)) then return null else { <u>delete and return</u> the front item of DQ }; // 데크의 rear에 있는 item(원소)을 데크에서 삭제하고 반환하는 연산 deleteRear(DQ) ::= if (isEmpty(DQ)) then return null else { <u>delete and return</u> the rear item of DQ }; // 데크의 front에 있는 item(원소)을 반환하는 연산 getFront(DQ) ::= if (isEmpty(DQ)) then return null else { return the front item of the DQ }; // 데크의 rear에 있는 item(원소)을 반환하는 연산 getRear(DQ) ::= if (isEmpty(DQ)) then return null else { return the rear item of the DQ }; End deque</pre>

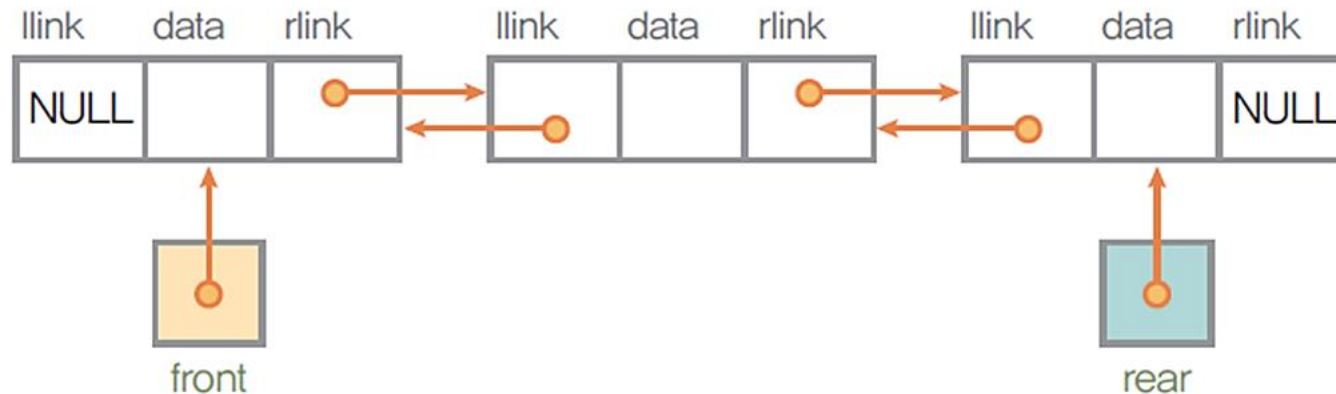
데크(deque)

❖ 데크의 연산과정



❖ 데크의 구현

- 양쪽 끝에서 삽입/삭제 연산을 수행하면서 크기 변화와 저장된 원소의 순서 변화가 많으므로 순차 자료구조는 비효율적임
- 양방향으로 연산이 가능한 이중 연결 리스트를 사용



❖ 이중 연결 리스트를 사용하여 데크를 구현

```
#define _CRT_SEQUERE_NO_WARNINGS
#include "stdio.h"
#include "stdlib.h"

// 데크 원소(element)의 자료형을 char로 정의
typedef char element;

// 이중 연결 리스트 데크의 노드 구조를 구조체로 정의
typedef struct DQNode {
    element data;
    struct DQNode *llink;
    struct DQNode *rlink;
} DQNode;

// 데크에서 사용하는 포인터 front와 rear를 구조체로 정의
typedef struct {
    DQNode *front, *rear;
} DQueueType;
```

❖ 공백 데크의 생성, 공백 상태 검사

```
// 공백 데크를 생성하는 연산
DQueueType *createDQueue()
{
    DQueueType *DQ;
    DQ = (DQueueType *)malloc(sizeof(DQueueType));
    DQ->front = NULL;
    DQ->rear = NULL;
    return DQ;
}

// 데크가 공백 상태인지 검사하는 연산
int isEmpty(DQueueType *DQ)
{
    if (DQ->front == NULL) {
        printf("\n Linked Queue is empty! \n");
        return 1;
    }
    else
        return 0;
}
```

❖ 데크의 front로 원소를 삽입하는 연산

```
// 데크의 front 앞으로 원소를 삽입하는 연산
void insertFront(DQueueType *DQ, element item)
{
    DQNode *newNode = (DQNode *)malloc(sizeof(DQNode));
    newNode->data = item;
    if (DQ->front == NULL) {           // 데크가 공백 상태인 경우
        DQ->front = newNode;
        DQ->rear = newNode;
        newNode->rlink = NULL;
        newNode->llink = NULL;
    }
    else {
        DQ->front->llink = newNode;
        newNode->rlink = DQ->front;
        newNode->llink = NULL;
        DQ->front = newNode;
    }
}
```

❖ 데크의 rear로 원소를 삽입하는 연산

```
// 데크의 rear 뒤로 원소를 삽입하는 연산
void insertRear(DQueueType *DQ, element item)
{
    DQNode *newNode = (DQNode *)malloc(sizeof(DQNode));
    newNode->data = item;
    if (DQ->rear == NULL) {                // 데크가 공백 상태인 경우
        DQ->front = newNode;
        DQ->rear = newNode;
        newNode->rlink = NULL;
        newNode->llink = NULL;
    }
    else {
        DQ->rear->rlink = newNode;
        newNode->rlink = NULL;
        newNode->llink = DQ->rear;
        DQ->rear = newNode;
    }
}
```


❖ 데크의 front 노드를 삭제하는 연산

```
// 데크의 front 노드를 삭제하고 반환하는 연산
element deleteFront(DQueueType *DQ)
{
    DQNode *old = DQ->front;
    element item;
    if (isEmpty(DQ))
        return 0;
    else {
        item = old->data;
        if (DQ->front->rlink == NULL) {
            DQ->front = NULL;
            DQ->rear = NULL;
        }
        else {
            DQ->front = DQ->front->rlink;
            DQ->front->llink = NULL;
        }
        free(old);
        return item;
    }
}
```

❖ 데크의 rear 노드를 삭제하는 연산

```
// 데크의 rear 노드를 삭제하고 반환하는 연산
element deleteRear(DQueueType *DQ)
{
    DQNode *old = DQ->rear;
    element item;
    if (isEmpty(DQ))
        return 0;
    else {
        item = old->data;
        if (DQ->rear->llink == NULL) {
            DQ->front = NULL;
            DQ->rear = NULL;
        }
        else {
            DQ->rear = DQ->rear->llink;
            DQ->rear->rlink = NULL;
        }
        free(old);
        return item;
    }
}
```

❖ 데크의 front 노드의 데이터를 반환하는 연산

```
// 데크의 front 노드의 데이터 필드를 반환하는 연산
element peekFront(DQueueType *DQ)
{
    element item;
    if (isEmpty(DQ))
        return 0;
    else {
        item = DQ->front->data;
        return item;
    }
}
```

❖ 데크의 rear 노드의 데이터를 반환하는 연산

```
// 데크의 rear 노드의 데이터 필드를 반환하는 연산
element peekRear(DQueueType *DQ)
{
    element item;
    if (isEmpty(DQ))
        return 0;
    else {
        item = DQ->rear->data;
        return item;
    }
}
```

❖ 데크의 원소들을 출력

```
// 데크의 front 노드부터 rear 노드까지 출력하는 연산
void printDQ(DQueueType *DQ)
{
    DQNode *temp = DQ->front;
    printf("DeQue : [");
    while (temp)
    {
        printf("%3c", temp->data);
        temp = temp->rlink;
    }
    printf(" ] ");
}
```

❖ 데크의 동작을 확인하는 메인 함수

```
void main(void)
{
    DQueueType *DQ1 = createDQueue(); // 데크 생성
    element data;
    printf("\n ***** 데크 연산 ***** \n");
    printf("\n front 삽입 A>> "); insertFront(DQ1, 'A'); printDQ(DQ1);
    printf("\n front 삽입 B>> "); insertFront(DQ1, 'B'); printDQ(DQ1);
    printf("\n rear 삽입 C>> "); insertRear(DQ1, 'C'); printDQ(DQ1);
    printf("\n front 삭제 >> "); data = deleteFront(DQ1); printDQ(DQ1);
    printf("\n\t삭제 데이터: %c", data);
    printf("\n rear 삭제 >> "); data = deleteRear(DQ1); printDQ(DQ1);
    printf("\n\t\t삭제 데이터: %c", data);
    printf("\n rear 삽입 D>> "); insertRear(DQ1, 'D'); printDQ(DQ1);
    printf("\n front 삽입 E>> "); insertFront(DQ1, 'E'); printDQ(DQ1);
    printf("\n front 삽입 F>> "); insertFront(DQ1, 'F'); printDQ(DQ1);

    data = peekFront(DQ1); printf("\n peek Front item : %c \n", data);
    data = peekRear(DQ1); printf("\n peek Rear item : %c \n", data);

    getchar();
}
```

데크(deque) 구현 프로그램

❖ 이중 연결 데이터 구조를 이용한 데크 구현 프로그램 : [교재 305p](#)

■ 실행 결과

```
C:\한국산업기술대학교\2021년1학기\데이터구조론\실습예제\큐(Deque)...  _  □  X

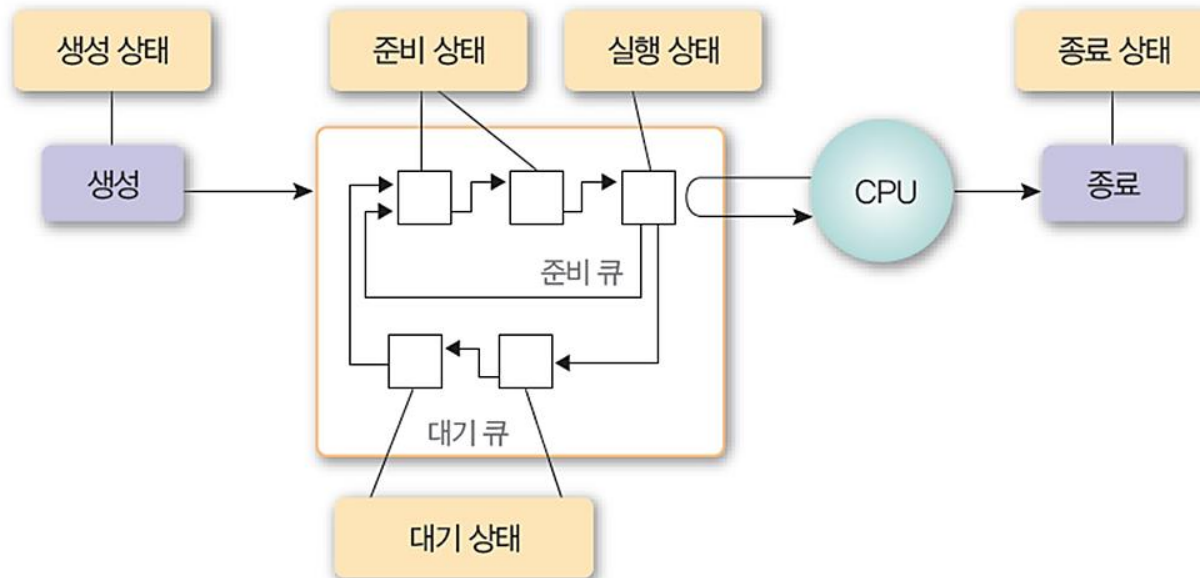
***** 데크 연산 *****

front 삽입 A>> DeQue : [ A ]
front 삽입 B>> DeQue : [ B A ]
rear 삽입 C>> DeQue : [ B A C ]
front 삭제 >> DeQue : [ A C ]      삭제 데이터 : B
rear 삭제 >> DeQue : [ A ]        삭제 데이터 : C
rear 삽입 D>> DeQue : [ A D ]
front 삽입 E>> DeQue : [ E A D ]
front 삽입 F>> DeQue : [ F E A D ]
peek Front item : F
peek Rear item : D

-----
Process exited after 7.424 seconds with return value 10
계속하려면 아무 키나 누르십시오 . . .
```

❖ 운영체제의 작업 큐

- 프린터 버퍼 큐(Printer Buffer Queue)
 - CPU에서 프린터로 보낸 데이터 순서대로(선입선출) 프린터에서 출력하기 위해 선입선출 큐 사용
- 스케줄링 큐(Scheduling Queue)
 - CPU 사용을 요청한 프로세서들의 순서를 스케줄링 하기 위해서 큐를 사용



❖ 시뮬레이션

- 시뮬레이션을 위한 수학적 모델링에서 대기행렬과 대기 시간 등을 모델링 하기 위해서 큐잉 이론(Queue theory) 사용
- 큐잉 모델은 고객에 대한 서비스를 수행하는 서버와 서비스를 받는 고객들로 이루어진다.
- 은행에서 고객이 들어와 서비스를 받고 나가는 과정을 시뮬레이션
 - 고객들이 기다리는 평균 시간을 계산



❖ 시뮬레이션 조건

- 시뮬레이션은 하나의 반복 루프
- 현재 시각을 나타내는 clock이라는 변수를 하나 증가
- is_customer_arrived 함수를 호출한다. is_customer_arrived 함수는 랜덤 숫자를 생성하여 시뮬레이션 파라미터 변수인 arrival_prob와 비교하여 작으면 새로운 고객이 들어왔다고 판단
- 고객의 아이디, 도착시간, 서비스 시간 등의 정보를 만들어 구조체에 복사하고 이 구조체를 파라미터로 하여 큐의 삽입 함수 enqueue()를 호출한다.

❖ 시뮬레이션 조건

- 고객이 필요로 하는 서비스 시간은 역시 랜덤숫자를 이용하여 생성된다.
- 지금 서비스하고 있는 고객이 끝났는지를 검사: 만약 `service_time`이 0이 아니면 어떤 고객이 지금 서비스를 받고 있는 중임을 의미한다.
- `clock`이 하나 증가했으므로 `service_time`을 하나 감소시킨다.
- 만약 `service_time`이 0이면 현재 서비스 받는 고객이 없다는 것을 의미한다. 따라서 큐에서 고객 구조체를 하나 꺼내어 서비스를 시작한다..

❖ 시뮬레이션 프로그램 만들기

```
#define _CRT_SECURE_NO_WARNINGS
#include "stdlib.h"
#include "stdio.h"
#include "math.h"

#define TRUE 1
#define FALSE 0
#define MAX_QUEUE_SIZE 100

typedef struct {
    int id;
    int arrival_time;
    int service_time;
} element;

typedef struct {
    element queue[MAX_QUEUE_SIZE];
    int front, rear;
} QueueType;

QueueType queue;
```

❖ 시뮬레이션 프로그램 만들기

```
// 시뮬레이션에 필요한 여러 가지 상태 변수
int duration = 10;           // 시뮬레이션 시간
double arrival_prob = 0.7;   // 하나의 시간 단위에 도착하는 평균 고객의 수
int max_serv_time = 5;       // 하나의 고객에 대한 최대 서비스 시간
int clock;

// 시뮬레이션의 결과
int customers;               // 전체 고객 수
int served_customers;        // 서비스 받은 고객 수
int waited_time;            // 고객들이 기다린 시간
```

❖ 시뮬레이션 프로그램 만들기

```
// 초기화 함수
void init(QueueType *q)
{
    q->front = q->rear = 0;
}

// 공백 상태 검출 함수
int is_empty(QueueType *q)
{
    return (q->front == q->rear);
}

// 포화 상태 검출 함수
int is_full(QueueType *q)
{
    return ((q->rear + 1) % MAX_QUEUE_SIZE == q->front);
}
```

❖ 시뮬레이션 프로그램 만들기

```
// 삽입 함수
void enqueue(QueueType *q, element item)
{
    if (is_full(q)) {
        fprintf(stderr, "큐가 포화상태입니다.\n");
        return;
    }
    q->rear = (q->rear + 1) % MAX_QUEUE_SIZE;
    q->queue[q->rear] = item;
}

// 삭제 함수
element dequeue(QueueType *q)
{
    if (is_empty(q)) {
        fprintf(stderr, "큐가 공백상태입니다.\n");
        exit(1);
    }
    q->front = (q->front + 1) % MAX_QUEUE_SIZE;
    return q->queue[q->front];
}
```

❖ 시뮬레이션 프로그램 만들기

```
// [0..1] 사이의 난수 생성 함수
double random()
{
    return rand() / (double)RAND_MAX;
}

// 랜덤 숫자를 생성하여 고객이 도착했는지 도착하지 않았는지를 판단
int is_customer_arrived()
{
    if (random() < arrival_prob)
        return TRUE;
    else
        return FALSE;
}
```


❖ 시뮬레이션 프로그램 만들기

```
// 새로 도착한 고객을 큐에 삽입
void insert_customer(int arrival_time)
{
    element customer;

    customer.id = customers++;
    customer.arrival_time = arrival_time;
    customer.service_time = (int)(max_serv_time * random()) + 1;
    enqueue(&queue, customer);
    printf("고객 %d이 %d분에 들어옵니다. 서비스시간은 %d분입니다.\n",
           customer.id, customer.arrival_time, customer.service_time);
}
```

❖ 시뮬레이션 프로그램 만들기

// 큐에서 기다리는 고객을 꺼내어 고객의 서비스 시간을 반환한다.

```
int remove_customer()
{
    element customer;
    int service_time = 0;

    if (is_empty(&queue))
        return 0;
    customer = dequeue(&queue);
    service_time = customer.service_time - 1;
    served_customers++;
    waited_time += clock - customer.arrival_time;
    printf("고객 %d이 %d분에 서비스를 시작합니다.  
대기시간은 %d분이었습니다.\n",
           customer.id, clock, clock - customer.arrival_time);
    return service_time;
}
```

❖ 시뮬레이션 프로그램 만들기

```
// 통계치를 출력한다.  
void print_stat()  
{  
    printf("서비스받은 고객수 = %d\\n", served_customers);  
    printf("전체 대기 시간 = %d분\\n", waited_time);  
    printf("1인당 평균 대기 시간 = %f분\\n",  
           (double)waited_time /served_customers);  
    printf("아직 대기중인 고객수 = %d\\n", customers - served_customers);  
}
```

❖ 시뮬레이션 프로그램 만들기

```
// 시뮬레이션 프로그램
void main()
{
    int service_time = 0;

    clock = 0;
    while (clock < duration) {
        clock++;
        printf("현재시각=%d\n", clock);
        if (is_customer_arrived()) {
            insert_customer(clock);
        }
        if (service_time > 0)
            service_time--;
        else {
            service_time = remove_customer();
        }
    }
    print_stat();
}
```